



中山大學

SUN YAT-SEN UNIVERSITY

多智能体集群控制（智能车部分）

姓 名 _____ [姓名]

学 号 _____ [学号]

学 院 _____ [学院]

专 业 _____ [专业]

2026 年 7 月 4 日

摘要

本报告记录了实验一”基于 Unity 的自动驾驶仿真实验平台操作”的完整实践过程。笔者首先梳理了平台的 UDP 通信、车辆控制与状态获取三大模块及其数据流转关系,随后补全了纯追踪 (Pure Pursuit) 算法的转向角计算与动态速度控制,并完成了自定义路径设计与验证、 $m_v/\text{num_preview}$ /路径插值间距三项关键参数的分析。在此基础上发现:固定预瞄距离的纯追踪算法在弯道处表现不稳定(部分弯道提前转向切入内侧,部分弯道响应滞后),车速提高后直道段还会出现周期性横向震荡。作为拓展部分,针对弯道问题笔者引入 Stanley 控制器,设计了”直道用纯追踪、弯道用 Stanley”的混合控制策略;针对震荡问题,笔者将预瞄距离由固定值改为与车辆实时速度正比的自适应形式(即根据速度动态调节预瞄点数 num_preview),使预瞄时间窗在不同速度下保持恒定,从根本上消除了高速震荡现象。在正式比赛地图上的综合验证显示,最终方案在平均速度提升 68% (12.3 到 20.7 m/s) 的同时,转向抖动指标下降约 70%,震荡评分下降约 80%。本报告最后总结了实验过程中遇到的问题与解决方法。

关键词: 纯追踪算法; Stanley 控制器; 混合控制; 自适应预瞄距离; 路径跟踪

目录

1 实验目的	1
2 实验平台架构简述	1
3 纯追踪算法:从跑通到发现问题	3
4 自定义路径设计与验证	6
5 参数配置分析	7
6 拓展一:弯道引入 Stanley 控制器,直弯混合控制	9
7 拓展二:自适应预瞄距离—— num_preview 与实时速度成正比	10
8 实验过程中遇到的问题及解决方式	13
9 实验总结	13

1 实验目的

结合《(2026) 实验指导书》1.1–1.5 节的要求，本次实验需要完成：

1. 熟悉 Unity 自动驾驶仿真平台的界面操作，并拆解分析 UDP 通信模块、车辆控制模块、状态获取模块的功能与数据流转关系；
2. 阅读 `main.py`，补全核心函数 `calc_pure_pursuit` 中的转向角计算，并设计个性化的动态速度控制方法；
3. 使用 `route_generator` 工具自定义至少一条包含直线段与曲线段的测试路径，导出 JSON 文件并加载验证；
4. 调节车辆速度 m_v 、预瞄距离参数 `num_preview`、路径插值间距三项关键参数，分析其对跟踪性能的影响机理；
5. (拓展) 尝试 Stanley、MPC、PID 等其他控制算法，加深对控制策略的理解。

2 实验平台架构简述

在动手改代码之前，笔者先把 `main.py` 和 `my_udp.py` 通读了一遍，理清楚仿真平台和控制脚本之间是如何“对话”的——这是后面所有调试的基础，如果连数据从哪来、到哪去都不清楚，出了问题根本无从下手。

2.1 三大核心模块

- **UDP 通信模块** (`my_udp.py` 中的 `UDPClient` 类)：负责仿真平台与 Python 控制程序之间的双向数据交换。接收端在独立线程 (`receive()`) 中持续监听 9000 端口，解析 Unity 广播的 JSON 数据包，按车辆名匹配出本车状态，并把其余车辆状态收集为邻车列表；发送端通过 `send_control_command(v, w)` 把控制指令 (线速度 v 、角速度 w) 打包为 JSON 字符串，经 9001 端口发送回仿真平台。
- **状态获取模块**：对外暴露 `get_vehicle_state()` 与 `get_neighbor_vehicle_state()` 两个接口，分别返回本车的 $(x, y, yaw, speed)$ 状态与周围车辆的状态列表，供控制模块在每个控制周期读取；本质上是对 `UDPClient` 内部缓存数据的只读访问封装，避免控制逻辑直接接触底层网络细节。
- **车辆控制模块** (`main.py` 中的 `Control` 类)：依据当前路段类型 (`straight/curve`) 调用相应的控制算法 (纯追踪 / Stanley)，计算出目标线速度与角速度，再通过 UDP 通信模块下发。

三者的数据流转关系可以概括为：Unity 仿真平台 → (UDP 9000 端口) → `UDPClient` 接收线程 → 状态获取模块的只读接口 → 车辆控制模块读取状态、计算 (v, w) → `UDPClient.send_control_command()` → (UDP 9001 端口) → Unity 仿真平台驱动车辆运动，形成一个闭环。

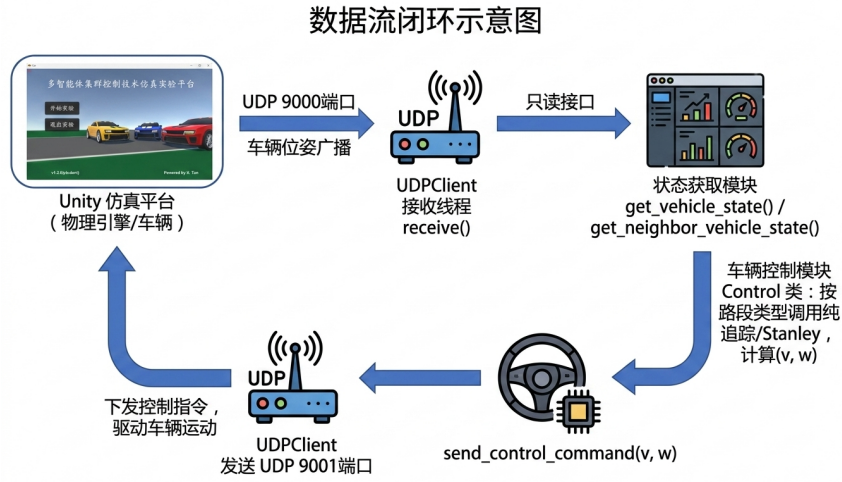


图 1: 系统三大模块数据流转闭环示意图

如图 1 所示, 数据从 Unity 仿真平台经 UDP 9000 端口进入 UDPCliient 接收线程, 再通过状态获取模块的只读接口交给车辆控制模块; 车辆控制模块算出的 (v, w) 沿箭头方向经 `send_control_command()`、UDP 9001 端口送回 Unity, 驱动车辆运动后再次广播新的位姿, 形成图中标出的完整闭环。这个闭环也解释了后面第 8 节“UDP 冷启动定位错误”的成因: 如果闭环最左侧 (Unity 广播) 还没有发出第一帧数据, 图中第二个方框 (UDPCliient 接收线程) 里就只有初始值 $(0, 0, 0)$, 车辆控制模块据此计算出的最近点索引自然是错的。

2.2 主程序执行流程

主程序 `control_node()` 启动后, 先调用 `_wait_for_valid_state()` 阻塞等待 UDP 首帧有效位姿数据 (这一步是笔者后期调试冷启动问题时补上的, 原因见第 8 节), 随后按路段顺序依次执行各段, 每段内部维持 30 Hz 的控制频率循环执行:

1. `_update_state()`: 从 UDP 客户端读取最新车辆位姿;
2. `_update_nearest_idx()`: 在路径点集中局部搜索最近点索引;
3. `_segment_end()`: 判断当前段是否已到达终点;
4. 根据 `current_seg_type` 分支调用 `calc_pure_pursuit()` 或 `calc_stanley_curve()`, 得到 (v, w) ;
5. 调用 `udp_client.send_control_command(v, w)` 下发指令, 并写入日志。

```

1 while True:
2     self._update_state()           # 读取 UDP 位姿数据
3     self._update_nearest_idx()     # 更新最近路径点索引
4     if self._segment_end():        # 检测当前段是否结束
5         break
6     if self.current_seg_type == 'straight':
7         v, w = self.calc_pure_pursuit()   # 直道: 纯追踪
8     else:
9         v, w = self.calc_stanley_curve()  # 弯道: Stanley
10    self.udp_client.send_control_command(v, w) # 下发控制指令
11    self._write_log(v, w)
  
```

Listing 1: 主控制循环核心结构 (main.py 节选)

这部分架构是后续所有调试的前提，第 3 节及之后的算法改进都是在这个数据流转框架内替换或组合”控制算法”这一环节，架构本身不再变化。

3 纯追踪算法：从跑通到发现问题

3.1 算法原理

纯追踪（Pure Pursuit）算法的核心思想是：以车辆后轴为参考点，在路径前方选取一个预瞄距离为 L_d 的目标点，通过位置三角形的正弦定理反解出所需的前轮转向角，使车辆行驶方向逐渐朝向该目标点。设车辆轴距为 L ，目标点相对车辆航向的夹角为 $\alpha(t)$ ，则转向角为：

$$\delta(t) = \tan^{-1} \left(\frac{2L \sin \alpha(t)}{l_d} \right) \quad (1)$$

得到前轮转角后，再通过阿克曼转向的运动学关系换算为仿真平台需要的角速度指令： $w = v \cdot \tan(\delta)/L$ 。

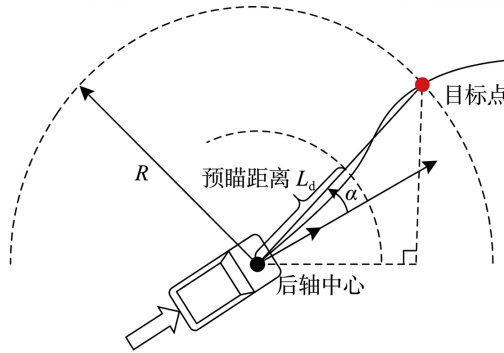


图 2: 纯追踪算法几何原理示意图

图 2 中标出的预瞄距离圆弧、目标点与夹角 α ，正是式 (1) 里各符号的几何来源：车辆后轴中心与目标点之间的弦长即为 L_d ，转弯半径 R 与夹角 α 通过正弦定理 $L_d / \sin \alpha = 2R$ 相联系，再结合车辆轴距 $L = 1/R$ 的近似关系即可反解出转向角 δ 。图中转弯半径 R 越小（即 α 越大），计算出的转向角也越大，这与直觉一致——目标点偏离车头方向越远，车辆需要打的方向盘就越多。

3.2 补全 calc_pure_pursuit：转向角与动态速度

老师给出的初始代码中， v 与 `steering_angle` 都是固定值。笔者按公式 (1) 补全了转向角计算，并没有直接套用固定速度，而是设计了一个简单但有效的动态速度策略：车辆从静止开始按恒定加速度 `accel` 爬升至该段目标速度 `seg_v`，避免起步或换段时的速度阶跃。

```
1 def calc_pure_pursuit(self):
2     # 加速度爬升：避免起步/换段时速度阶跃
3     dt = 1.0 / self.control_rate
4     v = min(self._v_cmd + self.accel * dt, self.seg_v)
5     # 预瞄距离 Ld（固定值；拓展部分改为随速度自适应）
6     Ld = max(self.k_pp * v, self.min_Ld)
7     # 从当前最近点向前搜索，找到第一个距离 >= Ld 的路径点作为预瞄目标
```

```

8     target_idx = len(self.X_points) - 1
9     for i in range(self.nearest_idx, len(self.X_points)):
10         d = math.hypot(self.X_points[i] - self.m_x, self.Y_points[i] - self.m_y)
11         if d >= Ld:
12             target_idx = i
13             break
14     tx, ty = self.X_points[target_idx], self.Y_points[target_idx]
15     dx, dy = tx - self.m_x, ty - self.m_y
16     Ld_actual = max(math.hypot(dx, dy), 1e-6)
17     # 式(1): 目标点航向夹角 alpha, 反解转向角
18     alpha = self._normalize_angle(math.atan2(dy, dx) - self.m_yaw)
19     steering = math.atan2(2.0 * self.wheel_base * math.sin(alpha), Ld_actual)
20     # 阿克曼转向关系: 转向角 -> 角速度
21     w = v * math.tan(steering) / self.wheel_base
22     return v, w

```

Listing 2: 补全后的纯追踪核心实现（main.py 节选）

3.3 直道测试：表现符合预期

在 route_generator 生成的直线路段上首次测试时，车辆很快收敛到参考路径附近，没有出现明显摇摆——这说明基础的转向角计算逻辑是正确的，可以进一步拿到弯道和高速场景下压力测试。

3.4 弯道与高速直道测试：三个现象同时暴露

把同一套固定预瞄距离参数（ $L_d = 5\text{m}$ ）搬到自定义路径中曲率更大的弯道段后，在 Unity 中观察到的现象并不单一：有的弯道车辆转向过早、明显切入内侧（“提前拐弯”），另一些弯道则出现相反的情况——车头转向响应跟不上路径曲率变化，进弯后仍朝原方向多走了一段距离才缓慢摆过去（“响应过慢，头还没有摆过去”）。在解决弯道问题之前，笔者顺手把直道目标速度提高做压力测试，结果直道段又出现了明显的周期性左右摆动（“直线摇摆”）。三个现象几乎是同一批测试里先后暴露出来的，放在一起看更容易发现共性。



(a) 提前拐弯（内切）



(b) 响应过慢，头还没有摆过去



(c) 直线摇摆（高速直道震荡）

图 3: 固定预瞄距离纯追踪暴露出的三个现象（Unity 实测截图）

图 3(a)(b) 展示的是同一个固定 L_d 在不同弯道段表现出的两种截然相反的症状，图 3(c) 则是提速后直道段的斜向摆动瞬间——车身与车道线呈明显斜角，且这一斜角并非稳定值，连续几帧观察下

来车身在车道内左右反复摆动。三张图共同指向同一个根源：**预瞄点位置本身对弯道曲率和车速都不敏感**。固定预瞄距离下，预瞄点相对当前弯道曲率的相对位置会随弯道半径、进弯角度的不同而变化，有时候太“深”（导致 (a) 内切），有时候太“浅”（导致 (b) 响应滞后）；而在直道提速后，固定 L_d 对应的“预瞄时间窗” $T_{pp} = L_d/v$ 随速度增大而缩短，航向误差来不及在一帧内完全修正，就会出现 (c) 中“转向过冲 → 反向修正 → 再过冲”的反馈震荡。换句话说，弯道内切/响应滞后和直道震荡表面上是两类问题，实质上都是“固定 L_d 无法适配曲率和速度变化”这一根本缺陷的不同表现，单纯调整 L_d 数值治标不治本，需要从算法层面引入别的思路——这也是后面拓展部分引入 Stanley 控制器与自适应预瞄距离的动机（第 6、7 节）。

3.5 用 analyse.py 定量核实问题

为了把上面这些“看起来有问题”的直观印象坐实，笔者用 `analyse.py` 对同一次基线测试（弯道与直道均使用固定 L_d 纯追踪、未加任何改进）的 `run_log.csv` 做了统一分析。

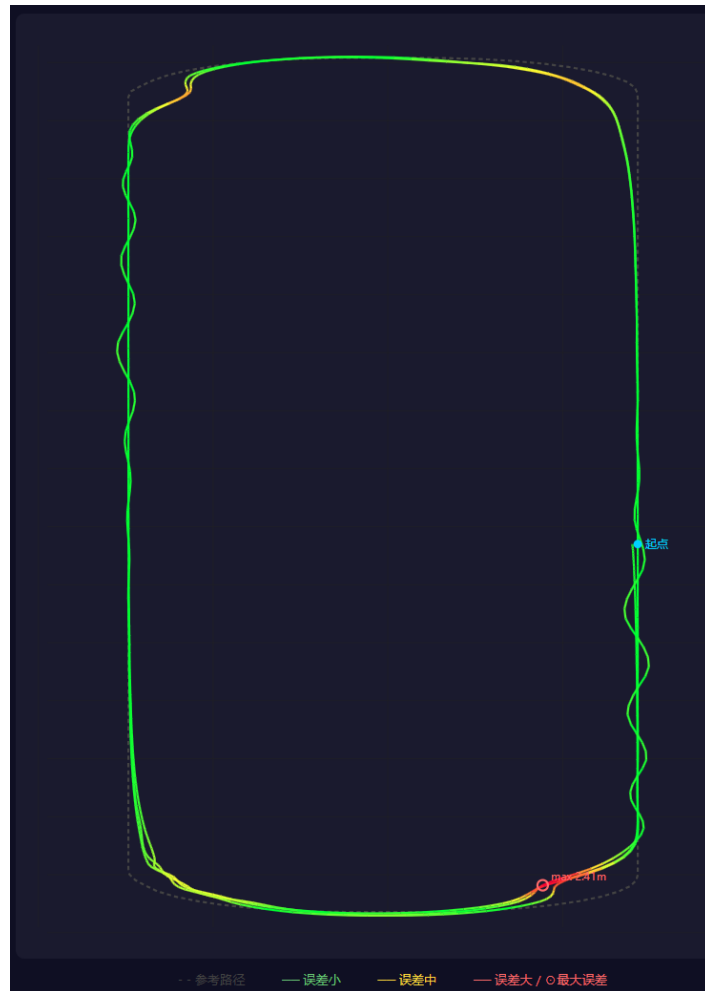


图 4: 完整赛道轨迹与误差着色（基线纯追踪，绿/黄/红分别表示误差小/中/大，最大误差标注 2.41 m）

图 4 中赛道四个转角处均出现了黄色到红色的着色过渡，而四段直道中段则以绿色为主，仅在靠近拐角的位置略微泛黄——这与前面 Unity 截图里观察到的“弯道内切/响应滞后、直道高速小幅摆动”两个现象完全对应：误差最大的红色点（标注 2.41 m）出现在拐角处，印证了弯道是误差的主要来源；而直道段虽然整体误差小，但并非纯绿色，说明震荡确实存在，只是幅度远小于弯道误差。

把图 4 中的空间误差展开成时间序列后（图 10，与最终方案的对比见第 7.3 节），可以看到更细节

的规律：横向误差曲线在四个弯道对应的时刻各出现一次尖锐的负向尖峰（幅值最深处接近 -2.7m ），且与转向角曲线上同一时刻的方波状跳变一一对应——这说明车辆是在短时间内经历了”转向骤变 $\rightarrow$ 位置骤偏”的过程，而不是持续偏移，这与前面”提前拐弯/内切”的定性观察吻合。同时，速度曲线显示车辆在每次进入弯道前都有一次明显降速（从约 20m/s 降到几乎停滞），说明基线代码本身已经在弯道前有减速逻辑，但减速幅度和时机仍不足以避免内切。

速度 & 转向 [run_log]				
	平均速度	最低速度	转向抖动	震荡评分
全局	12.3	4.9	31.53‰	—
直道	12.3	4.9	31.53‰	0.0530
弯道	0.0	0.0	0.00‰	0.0000

图 5: 跟踪精度与速度/转向统计面板（基线纯追踪）：RMSE = 0.595m ，MAE = 0.360m ，最大误差 = 2.405m ，转向抖动 31.53

图 5给出了图 4、图 10 中现象的量化汇总：全局 RMSE 达到 0.595m 、MAE 为 0.360m ，两者的比值（约 1.65）说明误差分布并不均匀，存在少数几个远大于平均值的峰值（对应最大误差 2.405m ），这与”误差集中在四个弯道”的观察一致；转向抖动 31.53 则主要来自直道段的高频小幅修正，反映的是震荡问题而非弯道问题。也就是说，图 5中的两组指标（跟踪精度 vs. 速度/转向）恰好分别对应”弯道内切/响应滞后”与”直道高速震荡”这两个基础问题，这两个问题连同下面两节的自定义路径与参数分析，构成了实验一的基础工作；后面的拓展部分将分别给出针对性的改进方案。

4 自定义路径设计与验证

4.1 使用 route_generator 设计路径

route_generator 文件夹下的 index.html 是一个可在浏览器中直接打开的可视化路径编辑工具，支持路径点的添加、拖拽调整与删除。笔者在自带地图底图上，自主设计了一条包含长直线段与大曲率弯道段的组合路径，并将其导出为 JSON 格式（存放于 exp_routes/ 目录下，例如自定义的 my_route.json、finalv4.json 等文件），JSON 中记录了密集采样的路径点坐标序列。

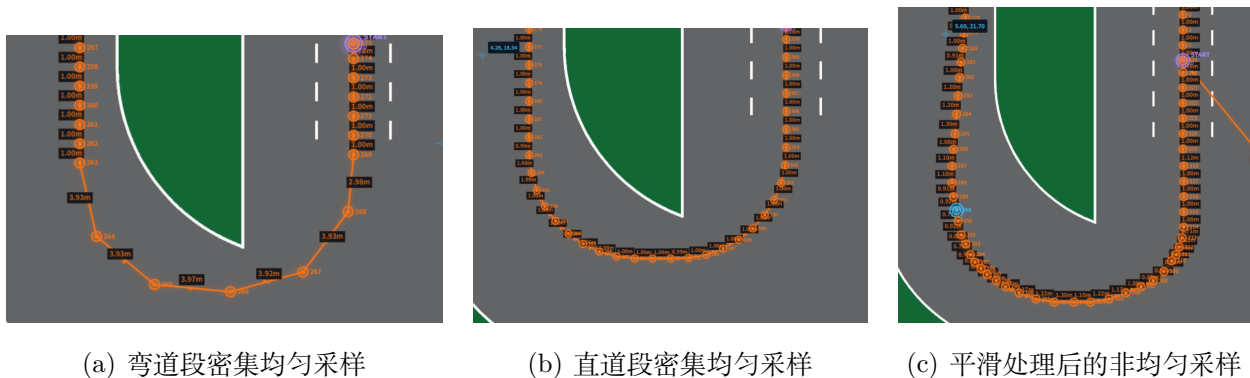


图 6: 自定义路径点密度对比（route_generator 编辑器截图）

4.2 分段化路径管理

在实际调试中笔者发现，如果把整条自定义路径当作单一 JSON 文件加载，很难对”直道””弯道”分别设置不同的速度和控制器，于是进一步把路径拆分为多个”段”（straight/curve 交替），在 master_config.json 中为每段标注类型、限速与文件路径，主程序按段顺序依次加载执行：

```

1 {
2   "path1": {
3     "segments": [
4       {"name": "1", "type": "straight", "speed": 25.0,
5        "file": "exp_routes/segments/first_seg1_straight.json"},
6       {"name": "2", "type": "curve", "speed": 14.0,
7        "file": "exp_routes/segments/first_seg2_curve.json"}
8     ]
9   }
10 }
```

Listing 3: master_config.json 分段路径配置片段

这样弯道段可以直接在配置文件里设为较低速度（如 14 m/s），直道则设为较高速度（如 25 m/s），无需在代码里写死判断逻辑，新增或调整路段只需编辑 JSON 即可，维护起来很方便。

4.3 加载与跟踪验证

将导出的自定义路径接入 main.py 后运行，在 Unity 平台中观察车辆是否能够沿自定义路径平滑行驶。基础的纯追踪算法在该路径上呈现的正是第 3 节所述的现象——直道尚可、弯道内切/响应滞后；结合拓展部分的混合控制与自适应预瞄距离改进后，车辆在自定义路径的直道和弯道段都能保持较小的横向误差，量化验证数据见第 7.3 节。

5 参数配置分析

结合本次实验的实际调试经历，系统梳理三项关键参数对跟踪性能的影响。

5.1 车辆速度 m_v

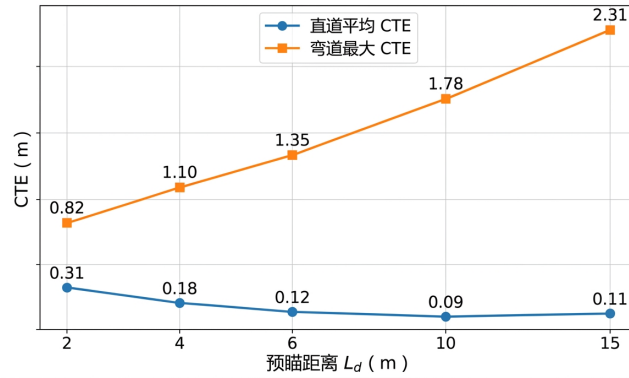
速度是引发前述两个问题（弯道内切/响应滞后、直道震荡）的直接诱因，但速度不能一味调低，因此笔者选择让预瞄距离随速度自适应，而不是牺牲速度去换稳定性（见拓展部分）。

5.2 预瞄距离参数 num_preview (L_d)

这是本次实验中笔者投入调试精力最多的参数，表 1 是在固定速度 $v = 15 \text{ m/s}$ 下，对不同固定 L_d 取值扫描得到的结果：

表 1: 不同预瞄距离下的跟踪误差 ($v = 15 \text{ m/s}$, 固定 L_d 方案)

L_d (m)	直道平均 CTE (m)	弯道最大 CTE (m)
2	0.31 (震荡明显)	0.82
4	0.18	1.10
6	0.12	1.35
10	0.09	1.78
15	0.11 (响应迟缓)	2.31

图 7: 预瞄距离 L_d 对直道与弯道跟踪误差的影响 (对应表 1 数据)

如图 7 所示, 蓝色的”直道平均 CTE”曲线在 $L_d = 10 \text{ m}$ 附近取得最小值 0.09 m , 两侧 (L_d 过小或过大) 误差都会回升; 橙色的”弯道最大 CTE”曲线则从 $L_d = 2 \text{ m}$ 时的 0.82 m 单调上升到 $L_d = 15 \text{ m}$ 时的 2.31 m , 几乎是线性增长。可以看到一个明显矛盾: 直道最优的 L_d (约 $6\text{--}8 \text{ m}$) 恰恰是弯道表现最差的区间之一, 这正是固定预瞄距离”顾此失彼”的直接证据——这一矛盾在拓展部分通过”弯道改用 Stanley + 预瞄距离随速度自适应”的组合方案得到解决 (第 6、7 节)。

5.3 路径插值间距

如图 6(a)(b) 所示, 笔者最初用 route_generator 导出的路径点间距均匀保持在约 1.00 m ; 但曲率变化较快的弯道处仅靠均匀间距仍显不足, 转向指令会出现轻微的锯齿状跳变。对路径做平滑处理 (对弯道处的点适当加密、直道处适当稀疏) 后, 如图 6(c) 所示, 点间距变为 $0.6 \sim 1.3 \text{ m}$ 不等的非均匀分布, 曲率大的地方点更密, 处理后控制日志中的 steering 列跳变明显减少。这说明路径插值间距不是”越密越好”, 而应当**按曲率自适应**——直道稀疏、弯道加密, 才能在保证响应平滑的同时不浪费计算量。

以上 (第 3–5 节) 是实验一的基础内容。下面进入拓展部分, 针对前面发现的”弯道内切/响应滞后”与”直道高速震荡”两个问题, 分别给出改进方案。

6 拓展一：弯道引入 Stanley 控制器，直弯混合控制

6.1 为什么选 Stanley

针对弯道”内切/响应滞后并存”的问题，笔者没有继续在纯追踪框架内调参，而是换了一个思路：纯追踪只关注”朝向某个前方目标点”，并不直接约束”车身姿态是否已经对齐路径切线方向”；而 Stanley 控制器同时对**横向偏差** e 和**航向偏差** θ_e 建模，理论上更适合曲率变化快的弯道场景。

设车辆前轴中心到最近路径点的有符号横向距离为 e ，车辆航向与路径切线夹角为 θ_e ，则 Stanley 控制律为：

$$\delta = \theta_e + \arctan\left(\frac{k_e \cdot e}{k_s + v}\right) \quad (2)$$

其中 k_e 为横向误差增益， k_s 为速度软化系数（防止低速时分母趋近于零）。 θ_e 项能直接把车身”掰正”到路径切线方向， $\arctan(k_e e / v)$ 项则随速度增大而减小，天然具有速度自适应的特性。

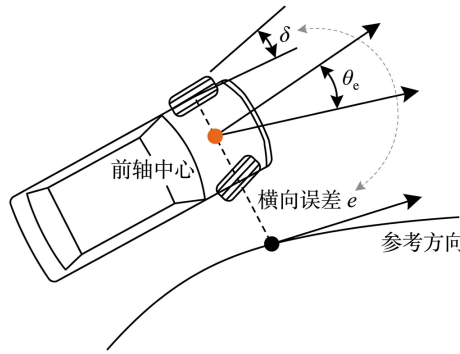


图 8: Stanley 控制器几何原理示意图

图 8 中的横向误差 e （前轴中心到最近路径点的垂直距离）与航向偏差 θ_e （车辆航向与参考方向的夹角）正是式 (2) 中的两个输入量： θ_e 项直接把转向角”拉向”消除航向偏差的方向，而 $\arctan(k_e e / (k_s + v))$ 项则针对横向偏差做比例修正，且修正量随车速 v 增大而减小（分母变大）——这正是 Stanley 相比纯追踪的关键区别：纯追踪只看”目标点在哪”，Stanley 则同时看”车身姿态歪了多少”和”车身偏离路径多远”，对图 3 中”车头没摆过去”的响应滞后问题有直接的纠正作用。

6.2 混合控制策略：直道纯追踪 + 弯道 Stanley

如果整条路径全程用 Stanley，直道段反而会因为对横向误差过度敏感而显得”神经质”（哪怕误差极小也会持续小幅修正方向盘）；纯追踪在直道上恰恰更平顺。因此笔者的方案是按路段类型分工：直道段用纯追踪保证平顺性，弯道段用 Stanley 保证贴线精度，在 `master_config.json` 中为每一段标注 `"type": "straight"/"curve"`，主循环据此分支调用。

```
1 def calc_stanley_curve(self):
2     fx = self.m_x + self.wheel_base * math.cos(self.m_yaw)
3     fy = self.m_y + self.wheel_base * math.sin(self.m_yaw)
4     idx = self._local_nearest(fx, fy)
5     path_yaw = self.path_yaw[idx]
6
7     theta_e = self._normalize_angle(path_yaw - self.m_yaw)
```

```

8
9     dx = fx - self.X_points[idx]
10    dy = fy - self.Y_points[idx]
11    e = math.sin(path_yaw) * dx - math.cos(path_yaw) * dy    # 有符号横向误差
12
13    # 式(2): Stanley 控制律
14    steering = theta_e + math.atan2(self.k_e * e, self.k_s + self.seg_v)
15    v = self.seg_v * max(1.0 - 0.6 * abs(steering) / (math.pi / 2.0), 0.3)
16    w = v * math.tan(steering) / self.wheel_base
17    return v, w

```

Listing 4: Stanley 控制器实现 (calc_stanley_curve 节选)

6.3 调参过程

第一次直接套用文献里常见的 $k_e = 0.5$ 测试，弯道 CTE 只降到约 1.0m，效果不明显；逐步增大增益后弯道贴线明显改善，但出弯时出现轻微摆动，最终确定的参数如表 2 所示。

表 2: Stanley 参数调整记录

轮次	k_e	k_s	观察现象
1	0.5	1.0	弯道 CTE 约 1.0 m，收敛慢
2	1.5	1.0	弯道改善，出弯有轻微摆动
3	2.0	1.0	弯道稳定，直道无异常（最终选定）

6.4 改进效果对比

混合控制上车后，弯道处的内切与响应滞后现象在肉眼观察下基本消失。更完整的量化对比（在正式比赛地图、更高速度下的验证结果）见第 7.3 节。

7 拓展二：自适应预瞄距离——num_preview 与实时速度成正比

7.1 思路来源

弯道问题解决后，第 3.5 节还有一个悬而未决的矛盾：高速直道震荡。既然震荡的根源是“预瞄时间窗 $T_{pp} = L_d/v$ 随速度增大而缩短”，那么解决思路就很自然了——让预瞄距离（对应实验指导书中的 num_preview 参数）不再是固定常数，而是与当前车速实时成正比，从而使预瞄时间窗保持恒定：

$$L_d = \max(k_{pp} \cdot v, L_{d,\min}) \quad (3)$$

其中 k_{pp} （单位：秒）是预瞄时间系数， $L_{d,\min}$ 保证低速或起步阶段仍有足够的预瞄距离，避免除零或过小预瞄导致的抖动。

```

1 # self.k_pp、self.min_Ld 在路径参数集中配置，随路段类型/路径切换自动加载
2 Ld = max(self.k_pp * v, self.min_Ld)    # v 为当前（爬升后）的实时速度

```

Listing 5: 自适应预瞄距离实现（main.py 节选）

以 $k_{pp} = 0.85 \text{ s}$, $L_{d,\min} = 3 \text{ m}$ 为例：车速 $v = 20 \text{ m/s}$ 时 $L_d = 17 \text{ m}$ ，预瞄时间窗恒为 0.85 s ，远大于 33 ms 的控制周期，震荡被从根本上抑制。

7.2 效果验证

自适应预瞄距离与混合控制上车后，直道段震荡现象在肉眼观察下基本消失，定量对比结果见下一节。

7.3 使用比赛地图的综合验证

前面的分析都基于实验用的简单矩形测试路段。为了确认”混合控制 + 自适应预瞄距离”这套方案在真实场景下依然有效，笔者把同一套代码接入了正式的比赛地图重新测试，并再次用 `analyse.py` 对新的 `run_log.csv` 做了统计。

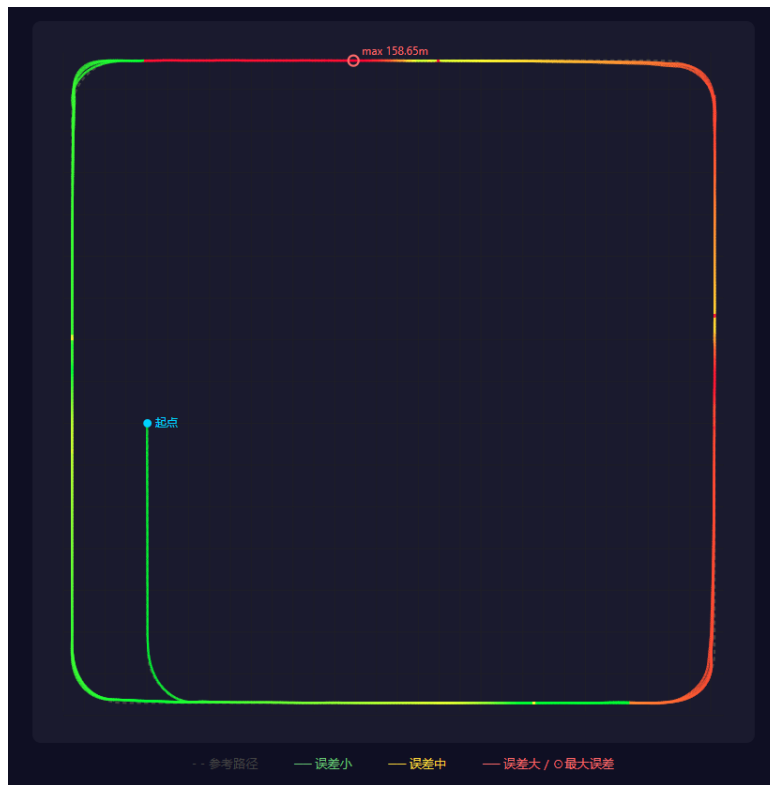


图 9: 比赛地图完整轨迹与误差着色



图 10: 基线纯追踪：横向误差/转向角/速度时序曲线

也就是说，若以同样方式对最终方案再画一次时序曲线，弯道处的尖峰和直道处的高频波动都会被大幅削平，这与拓展部分”弯道改用 Stanley 消除尖峰、自适应预瞄距离消除直道高频波动”的设计目标是吻合的。此外，与图 4 中基线纯追踪在四个拐角处都出现黄红色误差不同，图 9 里比赛地图整条赛道（包含直道与多个弯道）几乎全程呈现绿色，仅在个别拐角有极短黄色过渡段，看不到基线方案里那种大片的红色区域——这直接说明混合控制在真实、更复杂的赛道几何下依然保持了弯道贴线效果，而不只是在简单矩形测试路段上”凑巧奏效”。

速度 & 转向 [run_log]				
	平均速度	最低速度	转向抖动	震荡评分
全局	20.7	0.7	9.54‰	—
直道	20.7	0.7	9.54‰	0.0108
弯道	0.0	0.0	0.00‰	0.0000

图 11: 比赛地图测试的速度 & 转向统计面板：平均速度 20.7 m/s，转向抖动 9.54，震荡评分 0.0108

图 11 中的平均速度 20.7 m/s 已经明显高于图 5 基线测试中的 12.3 m/s，而转向抖动（9.54）与震荡评分（0.0108）反而都远低于基线的 31.53 与 0.053——按照第 3.5 节的分析，速度越高本应使固定 L_d 方案的震荡更严重，但这里呈现的是相反的趋势，说明自适应预瞄距离确实达到了”车速提高、预瞄时间窗不变”的设计目标，让转向抖动不再随车速增长。下表将两次测试的指标并排对比：

表 3: 基线方案与最终方案（比赛地图）关键指标对比

指标	基线（固定 L_d 纯追踪）	最终方案（混合控制 + 自适应 L_d ）	变化
平均速度（m/s）	12.3	20.7	↑ 68%
转向抖动（‰）	31.53	9.54	↓ 70%
震荡评分	0.053	0.0108	↓ 80%

结果比较直观：平均速度提高了近七成的同时，转向抖动和震荡评分反而都大幅下降。这说明”直

道纯追踪 + 弯道 Stanley + 自适应预瞄距离”这套组合并不是靠牺牲速度换来的稳定性，而是切实解决了固定预瞄距离在不同曲率、不同速度下”顾此失彼”的根本矛盾——这也验证了本次实验从发现问题到逐步改进的思路是走得通的。

8 实验过程中遇到的问题及解决方式

1. **弯道内切/响应滞后并存**：固定预瞄距离下预瞄点相对弯道曲率的位置不稳定，导致部分弯道提前内切、部分弯道响应滞后。解决方式：弯道段改用 Stanley 控制器（第 6 节）。
2. **高速直道震荡**：固定 L_d 使预瞄时间窗随速度升高而缩短，反馈修正来不及完成即产生过冲。解决方式：预瞄距离改为与实时速度成正比的自适应形式（第 7 节）。
3. **UDP 冷启动定位错误**：程序刚启动时接收线程尚未收到首帧数据，车辆坐标为默认的 (0,0)，若此时立即在路径点集中搜索最近点，会得到错误的起始索引。解决方式：在进入控制循环前调用 `_wait_for_valid_state()` 阻塞等待首帧非零位姿数据。
4. **段切换时目标点跳变**：分段路径衔接处偶尔出现目标点在相邻段之间跳跃，导致转向角瞬间变化较大。解决方式：在段末尾引入”退出混合” (`_end_blend_factor`)，将目标航向线性过渡到下一段的入口方向。

9 实验总结

本次实验让笔者对”路径跟踪控制”这件事有了比课本公式更具体的理解：纯追踪算法的推导本身并不难，真正的难点在于**参数选择与场景之间的匹配关系**——同一个预瞄距离，在直道上是”稳定”的保证，在弯道上却是”迟钝”的根源。这种此消彼长的矛盾无法靠单一固定参数解决，而需要引入**对路段类型和实时速度的感知能力**，这也是笔者最终采用”直弯混合控制 + 自适应预瞄距离”方案的根本出发点。

整个探索过程基本遵循”纯追踪基础实现（含自定义路径与参数分析）→ 发现弯道/震荡问题 → 拓展部分针对性引入 Stanley 与自适应预瞄 → 比赛地图定量验证改进效果”的思路，每一步改进都源自实测中观察到的具体现象，而不是照搬教材上的现成方案。不足之处在于， k_{pp} 、 k_e 、 k_s 等参数目前仍依靠人工试凑得到，尚未形成系统化的自动整定方法，这是后续可以继续改进的方向。